

Testautomatisering i github actions

- Lukas Alksäter
- Examensarbete 10 yhp
- Mjukvarutestare 2023

Innehållsförteckning

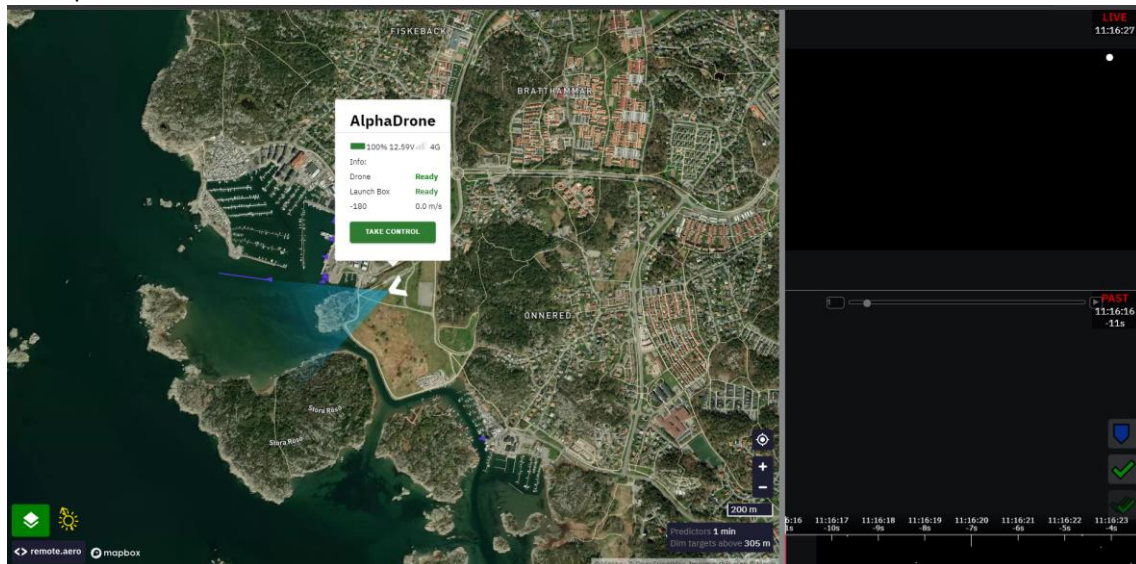
Innehållsförteckning	2
1. Introduktion	3
2. Syfte	4
3. Mål	4
4. Metod	4
5. Resultat.....	5
6. Slutsatser och diskussion	12
7. Referenser och källor.....	14

1. Introduktion

Detta examensarbete har utförts i samband med LIA (Lärande i arbete) på Remote Aero. Remote Aero jobbar med ett projekt som kallas EOS (Eyes on scene) vilket handlar om att drönare ska flygas med sjöräddningssällskapet för att ge en annan vinkel. Detta görs genom att drönaren skickar bild/video av platsen där olyckan hände till räddning sätningen. Remote Aero fokuserar på att styra drönaren.

Applikation är uppdelad 4 olika moduler. Den första är Backenden byggd i python vilket tar hand om autentisering och API som är byggt i Fastapi. Den tar också hand om websockets vilket är hur man autentiserar sig. Sedan finns det en frontend modul gjort i typescript som bygger självaste hemsidan

Bild på webbsidan när man valt drönare



På hemsidan kan man välja mellan olika drönare, och sedan välja en av drönaren och då kan man välja mellan pilot och spectate, om du väljer pilot ser du som bilden ovan, väljer man spectate försvinner take control delen biten och man kan bara kolla på och lägga till locations, medans som pilot kan man styra drönaren också.

Sedan finns det också en onboard modul som innehåller kod som körs på självaste drönaren. Sedan finns det också en sim modul som körs onboard koden på en simulerat drönare för testning och utveckling. Applikationen byggs med hjälp av docker och sedan byggs och pushas i en ci pipeline gjort i github actions.

Bild på en del av pipeline

```

1 name: build-and-push-images
2
3 on:
4   pull_request:
5     branches: [ "main" ]
6     types: [closed]
7
8
9
10 jobs:
11   backend:
12     if: github.event.pull_request.merged == true
13
14     env:
15       REGISTRY: gher.io
16       IMAGE_NAME: ${{ github.repository }}
17
18     permissions:
19       contents: read
20       packages: write
21
22     runs-on: ubuntu-latest
23
24
25     steps:
26     - name: Checkout repository
27       uses: actions/checkout@v3
28
29     - name: Set up Docker Buildx
30       uses: docker/setup-buildx-action@v1
31
32     - name: Log in to the Container registry
33       uses: docker/login-action@65b78e13532ed99fa3aa52ac7964289d1e9c1
34       with:
35         registry: ${{ env.REGISTRY }}
36         username: ${{ github.actor }}
37         password: ${{ secrets.GITHUB_TOKEN }}
38
39     - name: Build and push Backend Docker Image
40       uses: docker/build-push-action@v4
41       with:
42         context: ./backend

```

Projektet har utvecklats av flera olika studenter samt en fulltids utvecklare. Det ända testerna som sker i dagsläget är några få manuella tester

2. Syfte

Eftersom remote-mission-control ska styra riktiga drönare är det väldigt viktigt att webbsidan fungerar som den ska, eftersom om webbsidan slutar fungera mitt i en flygning och då kan drönaren till exempel krascha, eller köra åt fel håll, t.ex. i ett område där man inte får flyga. Därför behövs det tester för detta, och om det flesta är automatiserade märker man mycket tidigare om något blir fel.

3. Mål

Ta fram ett Proof-of-concept för att automatisera flera olika testfall och skapa en pipeline för att köra automatiskt för remote mission-control. Det behövs då skapas. Målet är att köra minst ett enhets test och integration test i en pipeline samt ska det vara möjligt att utöka pipeline för att stödja fler test typer/nivåer

4. Metod

Genom att arbeta i ett agilt team enligt Scrum-ramverket, tas proof-of-concept fram genom inkrementell utveckling. Konceptet utgår från att skapa en eller flera pipelines för att köra tester som ska skapas på företagets projekt. Verktögen för pipeline ska vara github actions, vilket Remote Aero redan använder, medan testverktygen väljs av teamet. Dessa valdes utefter

om det gick att testa python koden i backend och för integration testerna valdes det för teamet hade använt det innan

Pipeline ska byggas så att det ska gå att köra de tester som teamet tar fram med hjälp av produktägaren. Testerna tas fram efter diskussion med produktägaren och teamet av vad som är prioriterat men också det som var lättast att lära sig först. Därför ska det börjas med tester för backend. Sedan ska en pipeline skapats efter att några testfall har skapats, vilket gör att pipelines skapas efter behov. Pipelines ska vara separata från de redan existerande pipelines, efter önskemål av produktägare.

Pipelines ska försökas att ha en så kort exekveringstid som möjligt. Detta är på grund av att remote aero har ett visst antal bygg timmar som är gratis.

5. Resultat

Konceptet för pipelines har skapats i programmet github actions i form av 3 olika pipelines, en som kör lätta enhet tester vilka är skrivna i verktyget pytest. Den andra pipeline kör större tester gjorda i robotframework, som fokuserar på integrationen mellan backend och frontend på självaste hemsidan.

unit-tests pipeline kod

```

1  name: unit-tests
2
3  on:
4    pull_request:
5      branches: [ "main" ]
6    paths:
7      - 'backend/app/**'
8
9  jobs:
10   backend:
11     permissions:
12       contents: read #for fixing error with checkout
13       issues: read
14       checks: write
15       pull-requests: write #for comments
16
17     runs-on: ubuntu-latest
18
19     steps:
20       - name: Checkout Repository
21         uses: actions/checkout@v4
22
23       - name: Build Docker Container
24         working-directory: backend
25         run: docker build -t rmc-backend-test -f tests/Dockerfile .
26
27       - name: Run backend Tests
28         working-directory: backend
29         run: docker run --name backendtests -v $GITHUB_WORKSPACE/tests/results:/results rmc-backend-test
30
31       - name: Copy coverage file
32         run: docker cp backendtests:/results/coverage/html/ html
33
34       - name: Copy pytest results
35         run: docker cp backendtests:/results/pytest.xml pytest.xml
36
37       - name: Publish Coverage
38         uses: actions/upload-artifact@v4
39         with:
40           name: Results
41           path: html
42
43

```

Integration-tests pipeline har en mycket längre exekverings tid än den som inte startar hemsidan, detta är för att även med cache så tar det tid att bygga simuleringen av drönaren, samt att självaste testerna tar längre tid. Dock har exekverings tiden för starten av applikation gått från 21 minuter till 10 minuter med vilket inkluderar byggandet och laddningen av delarna tack vare

cacheing.

Exempel på unit test

```
def test_create_access_token():
    # Arrange: Create a test user with a username and role.

    current_user = auth.AuthUser(
        username="Tester", password="Fake Password", roles=[1000]
    )
    expectedUsername = current_user.username
    expectedRole = current_user.roles

    # Act: Make an access token for the user and decode it
    token = auth.create_access_token(current_user)
    result = jwt.decode(token, options={"verify_signature": False})

    # Assert: Check if the decoded token has the right username and role.
    assert result["username"] == expectedUsername
    assert result["roles"] == expectedRole
```

Integrations -test pipeline setup delen

```
10 jobs:
11   frontend:
12     permissions:
13       contents: read #for fixing error with checkout
14       issues: read
15       checks: write
16       pull-requests: write #for comments
17       packages: write
18
19   runs-on: ubuntu-latest
20   env:
21     REGISTRY: ghcr.io
22     IMAGE_NAME: ${github.repository}
23
24   steps:
25
26     - name: Remove unused Docker data
27       run: |
28         docker system prune -f
29         docker volume prune -f
30
31     - name: Free up space by removing system logs and caches
32       run: |
33         sudo rm -rf /usr/share/dotnet
34         sudo rm -rf /usr/local/lib/android
35         sudo rm -rf /opt/ghc
36         sudo apt-get clean
37         sudo rm -rf /var/lib/apt/lists/*
38
39     - name: Checkout Repository
40       uses: actions/checkout@v4
41       env:
42         DISPLAY: :99
43       |
44
45     - name: Set up Python
46       uses: actions/setup-python@v5
47       with:
48         python-version: '3.x'
```

```

49 - name: Install dependencies
50   run: |
51     python -m pip install --upgrade pip
52     pip install -r frontend/integration_tests/requirements.txt
53 - name: decode database
54   env:
55     DATA: ${secrets.DATABASE}
56   run: echo $DATA | base64 -di > backend/firebase-credentials.json
57 - name: decode drone credentials
58   env:
59     DRONE_CRED: ${secrets.DRONE_CREDENTIALS}
60   run: echo $DRONE_CRED | base64 -di > sim/credentials.json
61 - name: Create .env file
62   working-directory: backend
63   run: |
64     echo GOOGLE_APPLICATION_CREDENTIALS=firebase-credentials.json >> .env
65     echo 3MT_SECRET=Hbqc8UMZMv1ckfUu/bZD13x8Lb8Mh1waFRtB-J6Yj14- >> .env
66     echo 3ANUS_URL="http://localhost:8088/janus" >> .env
67
68 - name: Create .env.backend.local
69   working-directory: frontend
70   run: |
71     echo REACT_APP_MS_URI=ws://localhost:8000/api >> .env.backend.local
72     echo REACT_APP_HTTP_URI=http://localhost:8000/api >> .env.backend.local
73     echo REACT_APP_MAPBOX_TOKEN=${secrets.MAPBOX_TOKEN} >> .env.backend.local
74
75 - name: Set up Docker Buildx
76   uses: docker/setup-buildx-action@v1
77
78 - name: Log in to the Container registry
79   uses: docker/login-action@v2
80   with:
81     registry: ${env.REGISTRY}
82     username: ${github.actor}
83     password: ${secrets.GITHUB_TOKEN}
84
85

```

Integrations-test pipeline build och load delen

```

- name: Build Backend Docker Image
  uses: docker/build-push-action@v4
  with:
    context: ./backend
    file: ./backend/Dockerfile
    push: false
    tags: backend-dev
    cache-from: type=registry,ref=${env.REGISTRY}/${env.IMAGE_NAME}/backend:cache
    outputs: type=docker,dest=./backend-dev.tar

- name: Build Janus Docker Image
  uses: docker/build-push-action@v4
  with:
    context: ./backend/janus
    file: ./backend/janus/Dockerfile
    push: false
    tags: janus-dev
    cache-from: type=registry,ref=${env.REGISTRY}/${env.IMAGE_NAME}/janus:cache
    outputs: type=docker,dest=./janus-dev.tar

- name: Set up Node.js
  uses: actions/setup-node@v4
  with:
    node-version: 20

- name: Build Frontend Docker Image
  uses: docker/build-push-action@v4
  with:
    context: ./frontend
    file: ./frontend/Dockerfile-dev
    push: false
    tags: frontend-dev
    cache-from: type=registry,ref=${env.REGISTRY}/${env.IMAGE_NAME}/frontend-dev:cache
    cache-to: type=registry,ref=${env.REGISTRY}/${env.IMAGE_NAME}/frontend-dev:cache,mode=max
    outputs: type=docker,dest=./frontend-dev.tar

- name: Build Sittl Docker Image
  uses: docker/build-push-action@v4
  with:
    context: ./sim/sittl
    file: ./sim/sittl/Dockerfile
    push: false
    tags: sittl
    cache-from: type=registry,ref=${env.REGISTRY}/${env.IMAGE_NAME}/sittl:cache
    cache-to: type=registry,ref=${env.REGISTRY}/${env.IMAGE_NAME}/sittl:cache,mode=max
    outputs: type=docker,dest=./sittl.tar

- name: Load Janus
  run: docker load --input ./janus-dev.tar

- name: Load Backend
  run: docker load --input ./backend-dev.tar

- name: Load Frontend
  run: docker load --input ./frontend-dev.tar

- name: Load Mavlink
  run: docker load --input ./mavlink-send.tar

- name: Load Mavlink-router
  run: docker load --input ./mavlink-router.tar

- name: Load Onboard
  run: docker load --input ./video.tar

- name: Load Sittl
  run: docker load --input ./sittl.tar

```

Integration-test-pipeline starta och köra test delen

```

188 - name: Start application
189   uses: hoverkraft-tech/compose-action@v2.0.2
190   with:
191     compose-file: "compose-local-backend.yaml"
192
193 - name: Verify Container Statuses
194   run: docker ps # Lists all running containers
195
196 - name: Run test
197   env:
198     VALID_USERNAME: ${secrets.LOCAL_USERNAME}
199     VALID_PASSWORD: ${secrets.LOCAL_PASSWORD}
200   run: |
201     python -m robot --outputdir $GITHUB_WORKSPACE/results --variable url:http://localhost:3000/ --variable valid_username:$VALID_USERNAME --variable valid_password:$VALID_PASSWORD frontend/integration_tests/
202
203 - name: Publish Results
204   uses: actions/upload-artifact@v4
205   if: success() || failure()
206   with:
207     name: Integration_Test
208     path: results/output.xml
209

```

Exempel på robotframework testfall

```

1  *** Settings ***
2  Documentation    EOS
3  Library          SeleniumLibrary
4  Library          Collections
5  Resource         resources/commonResources.resource
6  Resource         resources/logOutResources.resource
7  Test Setup       Test setup
8  Test Teardown    Test teardown
9
10 *** Test Cases ***
11 Scenario: user logs in by using wrong password
12   [Documentation]    User logs in with wrong password
13   [Tags]            login
14   Given The website opens correctly
15   When The user enters wrong password
16   Then The user fails to login
17
18 Scenario: user logs in with correct password and wrong username
19   [Documentation]    User logs in with wrong username
20   [Tags]            login
21   Given The website opens correctly
22   When The user enters wrong username
23   Then The user fails to login
24
25 Scenario: User logs in with valid credentials
26   [Documentation]    user logs in with correct username and password
27   [Tags]            login
28   Given The website opens correctly
29   When The user enters valid credentials
30   Then The user should be logged in
31
32 Scenario: User logs out and confirms they've logged out
33   [Documentation]    User after having logged in, then logs out.
34   [Tags]            log out
35   Given The website opens correctly
36   And The user enters valid credentials
37   And The user should be logged in
38   When The user is logged in
39   And The user clicks the log out button
40   Then The user confirms they've logged out

```

Exempel på robotframework test kod

```

35 The website opens correctly
36   Page Should Contain Element    ${logo}
37
38 The user enters wrong password
39
40   Wait Until Page Contains Element    ${logo}
41   Input Text    ${login_username_field}    ${valid_username}
42   Input Password    ${login_password_field}    ${invalid_password}
43
44 The user enters wrong username
45
46   Wait Until Page Contains Element    ${logo}
47   Input Text    ${login_username_field}    fake
48   Input Password    ${login_password_field}    ${valid_password}
49
50 The user fails to login
51   Click Button    ${login_submit_button}
52   Page Should Contain Element    ${errorMessage}
53
54 The user enters valid credentials
55   Wait Until Page Contains Element    ${login_username_field}    timeout=15s
56   Input Text    ${login_username_field}    ${valid_username}
57   Wait Until Page Contains Element    ${login_password_field}    timeout=15s
58   Input Password    ${login_password_field}    ${valid_password}
59   Wait Until Page Contains Element    ${login_submit_button}    timeout=15s
60
61 The user should be logged in
62   Click Button    ${login_submit_button}
63   Wait Until Page Contains Element    ${mission_container_div}    timeout=15s

```

Både unit-tests och integration-tests pipelines publicerar sedan resultatet till pull requesten som aktiverade pipelinen, men integration testerna visar bara test som misslyckas, för att inte göra pull requests för röriga. Unit-tests

pipeline körs bara när en pull request skapas som ändrar i backend

```
on:
  pull_request:
    branches: [ "main" ]
    paths:
      - 'backend/app/**'
```

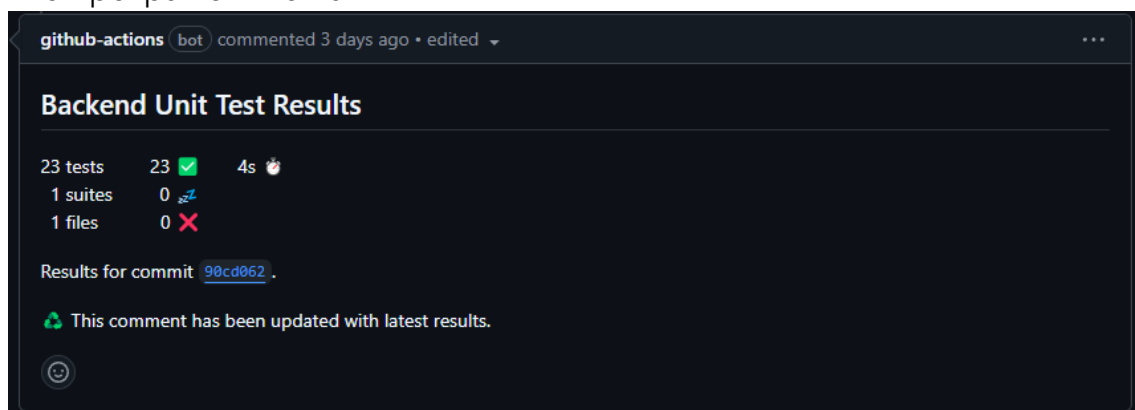
Integration-tests pipeline körs istället på när någonting ändras, förutom .yaml filer och .md filer.

```
on:
  pull_request:
    branches: [ "main" ]
    paths-ignore:
      - '**/*.md'
      - '**/*.yaml'
```

Båda av dessa pipelines publicerar också resultaten på olika sätt. Unit-test pipeline publicerar resultatet som en kommentar oavsett resultat

```
- name: Publish Test Results
  uses: EnricoMi/publish-unit-test-result-action@v2.17.1
  if: success() || failure()
  with:
    fail_on: "test failures"
    files: |
      pytest.xml
    check_name: "Backend Unit Test Results"
```

Exempel på kommentar



The screenshot shows a GitHub Actions workflow comment. At the top, it says 'github-actions bot commented 3 days ago • edited'. The title of the comment is 'Backend Unit Test Results'. Below the title, there is a summary of test results: '23 tests' (23 green checkmarks), '1 suites' (0 red X), and '1 files' (0 red X). The time taken is '4s'. Below this, it says 'Results for commit 90cd062'. At the bottom, there is a message: 'This comment has been updated with latest results.' with a refresh icon.

Integration-tests pipeline publicerar bara testresultatet som en kommentar om något av testen misslyckas, annars blir det bara skickat till pipelinesammanfattning

Exempel på kommentar:

github-actions bot commented 3 days ago • edited

Robot Results

Passed	Failed	Skipped	Total	Pass %	Duration
10	1	0	11	90.91	10m56.649157s

Failed Tests

Name	Message	Duration	Suite
Scenario: User increases speed of drone	Element '//button[normalize-space()='Take off']' not visible after 5 seconds.	64.572 s	Drone

Exempel på sammanfattning:

frontend summary

Robot Results

Passed	Failed	Skipped	Total	Pass %	Duration
11	0	0	11	100	12m56.752434999s

Job summary generated at run-time

Dessutom så tas kommentaren bort om testen misslyckades först och sedan lyckades

Integration-test kod för publicering av resultat

```

- name: Download reports
  uses: actions/download-artifact@v4
  if: success() || failure()
  with:
    name: Integration_Test
    path: Integration_Test
- name: Send report to commit
  if: failure()
  uses: joonvena/robotframework-reporter-action@v2.5
  with:
    gh_access_token: ${{ secrets.GITHUB_TOKEN }}
    report_path: Integration_Test
    show_passed_tests: false
- name: Find comment
  if: success()
  uses: peter-evans/find-comment@v3
  id: fc
  with:
    issue-number: ${{ github.event.number }}
    body-includes: Robot Results
- name: Delete comments
  if: success()
  uses: detomarcio/delete-comments@1.1.0
  with:
    token: ${{ secrets.GITHUB_TOKEN }}
    comment-id: ${{ steps.fc.outputs.comment-id }}
- name: Send report to summary
  if: success()
  uses: joonvena/robotframework-reporter-action@v2.5
  with:
    gh_access_token: ${{ secrets.GITHUB_TOKEN }}
    report_path: Integration_Test
    show_passed_tests: false
    only_summary: True

```

Som nämntes i början av detta kapitel finns det tre pipelines. Den sista kollar bara formatering och linting på backend kod just nu.

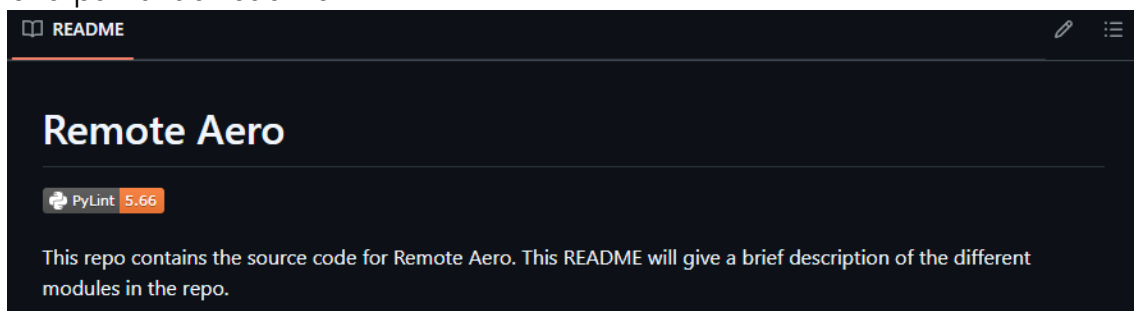
Bild på format pipeline kod

```

1  name: format
2
3  on:
4    pull_request:
5      branches: [ "main" ]
6      paths:
7        - '**/*.py'
8
9  jobs:
10
11    backend:
12      permissions:
13        contents: write #for fixing error with checkout
14        issues: read
15        checks: write
16        pull-requests: write #for comments
17
18      runs-on: ubuntu-latest
19
20      steps:
21        - name: Checkout Repository
22          uses: actions/checkout@v4
23        - name: Black format checker
24          uses: psf/black@stable
25          with:
26            options: "--check --diff --verbose --color"
27            src: "backend/app"
28
29        - name: Run pylint
30          if: success() || failure()
31          uses: Sliellie/pylint-github-action@v2
32          with:
33            lint-path: backend/app
34            python-version: 3.x
35            requirements-path: backend/tests/requirements.txt

```

Denna kollar bara om .py filer är formaterade med black formatting och sedan kör pylint på all python kod. Pylint resultatet publiceras sedan som en bild på huvud readme.



6. Slutsatser och diskussion

Däremot skulle integration testerna kunna skrivas med ett annat verktyg. Detta är eftersom vi inte hittade ett sätt högre klicka med verktyget, vilket behövs för att testa en hel del av remote aero hemsidan när man ska styra drönaren, eftersom robotframework beror på element och på kartan kunde robotframework inte hitta element, vilket gjorde att det inte gick att testa att styra drönaren, hastighets knaparna eller höjden. Dock är robotframework ett tydligt sätt att skriva testfall samt hade teamet redan arbetat med robotframework tidigare. Det gjorde att flera tester gick att göras snabbt eftersom det inte behövde ta tid att lära sig robotframework. Det går nog få till att högerklicka med robotframework men det har inte hittats ett sätt ännu.

Github actions är relevant för att det var verktyget som remote aero använde innan för sina build pipelines. Github actions har många limitationer, t.ex. så kan man inte specificera att ett visst jobb ska bara aktiveras om en del av projektet ändras, utan man kan bara specificera för hela pipeline, vilket gör att om man vill ha t.ex. backend och frontend enhet tester i samma pipeline

måste bådas köras t.ex. när backend/app ändras eller frontend/src ändras, istället för att bara en kör. Man skulle kunna ha en pipeline för varje del men det skulle bli många pipelines. Men förutom det kan man anpassa github actions för det man behöver, tack vare att användare kan skapa tillägg för pipelines. Detta gjorde att det gick att få kommentarer av testresultat till pull request, men också ta bort kommentarer och hitta rätt kommentar. Integration-tests pipeline skulle bara kommentera resultat om något misslyckades, efter feedback från produktägaren. Detta gjorde att man var tvungen att hitta tillägg som kunde ta bort kommentarer samt att rapportfunktionen hade en inställning för att bara skicka resultatet till pipelinesammanfattning. Utan dessa skulle det inte vara möjligt att skapa dessa pipelines.

Fördelar med dessa pipeline är att de bara kör när det ändras något som testerna tester eller i format pipeline bara när .py filer ändras. Men på grund av när github actions kollar vilken fil/folder ändras så kommer pipelines vara tvungen att köras alla jobb även om det den testar inte har ändrats, detta är mest uppenbart i unit-test pipeline där om man vill lägga till frontend enhet tester kommer man behöva köra backend tester även när frontend ändras och tvärtom. Det är samma sak med format pipelinen som kommer också behöva köra jobb som inte borde köras om man lägger till t.ex. linting för typescript. Däremot har dessa pipeline väldigt låg exekvering tid vilket gör att man inte förlorar så många bygg timmar ändå.

Integration-tests pipeline skadas inte av detta problem eftersom integration tester ska köras om någon del ändras, därför använder den av path-ignore istället för path, vilket betyder att den inte kör om några av dessa filer bara ändras, tillskillnad från path som betyder att den bara kör om någon fil i denna sökväg ändras. För vidareutveckling av integration-tests pipeline beror det på hur github actions sparar saker. Om man vill lägga till backend integration tester och men man måste starta hela applikationen igen, behöver man då köra hela build och load delen samt starta applikationen igen, eller sparas det vilket gör att du bara kan köra docker-compose kommande och det tar väldigt kort tid eftersom det redan byggts i jobbet innan.

Fördelar med att det gick att rapportera resultat till pull request samt att det räknas som en check att alla ska passera innan man kan göra en merge så minskar det risken för att buggar kommer till huvud branschen, vilket är väldigt viktigt eftersom denna applikation ska användas i liv och nödsituationer i framtiden och då vill man inte att buggar ska hittas. Därför går det inte att göra en merge utan att testerna går igenom korrekt. Däremot på grund av buggar när man kör robotframework i pipelines kan testerna misslyckas för anledningar som inte har med någon kod att göra, Samma sak

om man bara döper om något element som användas så misslyckas tester också, vilket betyder att man behöver ändra testerna.

Men överlag så är testerna och speciellt pipelines väldigt användbara för remote aero och det hjälper dem att hitta fel tidigt i projektet och det går att lägga till saker utan att ändra på det existerande koden. Däremot finns det risk att med uppdateringar av github actions att tilläggen som används kan sluta fungera men oftast uppdateras tilläggen för till en ny version. Så det viktigaste för remote aero är att hålla koll på uppdatering av github actions och tilläggen som används.

7. Referenser och källor

Tillägg unit-test pipeline:

- <https://github.com/actions/upload-artifact/tree/v4/>
- <https://github.com/EnricoMi/publish-unit-test-result-action/tree/v2.17.1/>

Tillägg format pipeline:

- <https://github.com/psf/black/tree/stable/>
- <https://github.com/Silleellie/pylint-github-action/tree/v2/>

Tillägg integration-tests pipeline:

- <https://github.com/actions/setup-python/tree/v5/>
- <https://github.com/docker/setup-buildx-action/tree/v1/>
- <https://github.com/docker/login-action/tree/v2/>
- <https://github.com/docker/build-push-action/tree/v4/>
- <https://github.com/actions/setup-node/tree/v4/>
- <https://github.com/docker/setup-qemu-action/tree/v1/>
- <https://github.com/hoverkraft-tech/compose-action/tree/v2.0.2/>
- <https://github.com/actions/download-artifact/tree/v4/>
- <https://github.com/joonvena/robotframework-reporter-action/tree/v2.5/>
- <https://github.com/peter-evans/find-comment/tree/v3/>
- <https://github.com/detomarco/delete-comments/tree/1.1.0/>